Edge cases (a missing required argument) — Hints

> Back to the exercise · Hint 1 · Hint 2 · Solution

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint builds toward the full solution.

- Hint 1 — The idea, and the exact pieces
- Hint 2 — The plan, and almost the whole thing

Hint 1 — The idea, and the exact pieces

Reuse a tool from an earlier exercise (`get_weather(city: str)` works fine) rather than building a new one — the point here is the *prompt*, not the tool.

Run the same ambiguous prompt (something like "what's the weather like") several times, and log, for each run: did the model call the tool at all, and if so, what value did it send for `city`? A model might send an empty string, a guessed default city, or genuinely refuse the call and respond with plain text asking you for the city.

Look specifically at **`response.content` vs. `response.tool_calls`:** when the model refuses to call the tool and asks you a question instead, that question lands in `response.content`, and `response.tool_calls` will be empty — check both every time, don't assume one or the other. Three buckets are enough to classify each run: "asked a clarifying question," "called the tool with a guessed/empty value," and "called the tool with something else unexpected."

This is exactly why "checking arguments" (Core Concepts) isn't the whole story: Pydantic only checks that a `city` value *exists and is a string* — it can't tell you whether the model quietly guessed "London" as a default. That's a values problem, not a shape problem, and it's worth seeing the difference here. Write your classification function's signature before moving to Hint 2.

Hint 2 — The plan, and almost the whole thing

```
# missing_argument_practice.py
def classify_response(response) -> str:
    if not response.tool_calls:
        return "asked_or_answered_directly"
```

```
    city_sent = response.tool_calls[0]["args"].get("city")
    if not city_sent:
        return "called_with_empty_value"
    return "called_with_guessed_value"
```

Wire this into a 5-run loop yourself, printing each run's outcome (and the model's reply when it asked instead of calling).

Then fix it at the tool itself: a Pydantic model with `city: str = Field(min_length=1)`, attached with `@tool(args_schema=WeatherArgs)`. Now `get_weather.invoke({"city": ""})` raises `ValidationError` — catch it by name and return `"Error: a city name is required - ..."`. Compare against the Solution.

Full solution: Show me the solution